

AI-Powered Legal Intelligence System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Animesh Parashar	-	EN22CS301128
Anirudh Singh Rathore	-	EN22CS301130
Anshika Parsai	-	EN22CS301151
Anshul Goyal	-	EN22CS301157
Anushka Mukati	-	EN22CS301181

Under the Guidance of
Prof.(Dr.) Hemlata Patel
Prof. Ajeet Singh Rajput
Prof. Akshay Saxena



MEDICAPS
U N I V E R S I T Y

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

AI-Powered Legal Intelligence System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING**

BY

Animesh Parashar	-	EN22CS301128
Anirudh Singh Rathore	-	EN22CS301130
Anshika Parsai	-	EN22CS301151
Anshul Goyal	-	EN22CS301157
Anushka Mukati	-	EN22CS301181

Under the Guidance of
Prof.(Dr.) Hemlata Patel
Prof. Ajeet Singh Rajput
Prof. Akshay Saxena



MEDICAPS
U N I V E R S I T Y

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work “**AI-Powered Legal Intelligence System**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name: Prof. Akshay Saxena

Prof. Hemlata Patel

Prof. Ajeet Singh Rajput

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled “**AI-Powered Legal Intelligence System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications in ‘**Computer Science and Engineering**’ completed under the supervision of **Prof. Ajeet Singh Rajput, Prof. Akshay Saxena, Dr. Hemlata Patel**, Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Animesh Parashar - EN22CS301128
Anirudh Singh Rathore - EN22CS301130
Anshika Parsai - EN22CS301151
Anshul Goyal - EN22CS301157
Anushka Mukati - EN22CS301181

Certificate

We, **Prof. Ajeet Singh Rajput, Prof. Akshay Saxena, Dr. Hemlata Patel** certify that the project entitled “**AI-Powered Legal Intelligence System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications by **Animesh Parashar, Anirudh Singh Rathore, Anshika Parsai, Anshul Goyal, Anushka Mukati** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

<Name of Internal Guide>

<Name of the Department>

Medicaps University, Indore

<Name of External Guide (If any)>

<Name of the Department>

Name of the Organization

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore











Acknowledgements

I would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean, Faculty of Engineering, Mediacaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

I express my heartfelt gratitude to my **External Guide**, as well as to my Internal Guide, **Prof. Ajeet Singh Rajput, Prof. Akshay Saxena, Dr. Hemlata Patel**, without whose continuous help and support, this project would ever have reached to the completion.

I would also like to thank to my team at **Animesh Parashar, Anirudh Singh Rathore, Anshika Parsai, Anshul Goyal, Anushka Mukati** who extended their kind support and help towards the completion of this project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Animesh Parashar –

EN22CS301128

Anirudh Singh Rathore –

EN22CS301130

Anshika Parsai –

EN22CS301151

Anshul Goyal –

EN22CS301157

Anushka Mukati –

EN22CS301181

B.Tech IV Year

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Abstract

The rapid growth of digital information has made manual research increasingly time-consuming and inefficient. This project presents the design and implementation of **an AI-Powered Legal Intelligence System** that automates the process of web-based research, analysis, and report generation.

The system leverages **Generative Artificial Intelligence (GenAI)** and **Agentic AI** architecture, where multiple specialized agents collaborate to perform tasks such as web searching, content scraping, data analysis, and report summarization. The application integrates modern technologies including React (Vite) for frontend development, Node.js (Express) as an API gateway, and FastAPI (Python) for backend processing. Advanced AI models such as HuggingFace Transformers (Flan-T5) and Sentence Transformers are used for natural language processing and semantic search.

The system also incorporates **FAISS** (Facebook AI Similarity Search) for efficient vector-based retrieval and uses **JSON-based storage** for maintaining research history. To ensure scalability and real-world applicability, the application is containerized using **Docker** and deployed on **AWS EC2** with a fully automated **CI/CD** pipeline using **GitHub Actions**.

The developed system successfully automates the research workflow, reduces manual effort, and generates structured reports from user queries. The project demonstrates the integration of AI, web technologies, and DevOps practices into a unified, scalable, and production-ready system.

Keywords:

S.No.	Keyword	Description
1	Generative AI	AI systems that generate content such as text, images, or responses using models like LLMs
2	Agentic AI	AI systems capable of autonomous decision-making and task execution
3	Legal Chatbot	AI-powered chatbot designed to interact with and answer queries related to legal documents
4	Autonomous Agents	Intelligent agents that perform tasks independently without continuous human intervention
5	DevOps	A set of practices combining software development and IT operations
6	CI/CD	Continuous Integration and Continuous Deployment pipeline for automating software delivery
7	Docker	Containerization platform used to package applications and dependencies
8	Ruby on Rails	Web application framework used for backend development

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii - viii
	List of figures	ix
	List of tables	x
	Abbreviations	xi
Chapter 1	Introduction (Whichever is applicable)	1-3
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	2
	1.5 Research Design	2
	1.6 Source of Data	2
	1.7 Chapter Scheme	3
Chapter 2	(Report on Present Investigation)	4-7
	2.1 System Architecture	4
	2.2 Technology Stack	6
	2.3 System Workflow	6
Chapter 3	Implementation	8-10
	3.1 Frontend Implementation	8
	3.2 Backend Implementation	8
	3.3 AI Agent Implementation	9
Chapter 4	DevOps & Deployment	11-14
	4.1 Dockerization	11
	4.2 CI/CD Pipeline (GitHub Actions)	12
	4.3 AWS EC2 Deployment	13
Chapter 5	Database & Storage	15-16
	5.1 FAISS Vector Database	15

	5.2 JSON Storage System	16
	5.3 Data Flow Management	16
Chapter 6	Results and Discussions	17-20
	6.1 System Output	17
	6.2 Performance Analysis	20
	6..3 Challenges Faced	20
Chapter 7	Summary and Conclusions	21
	7.1 Summary	21
	7.2 Conclusion	21
Chapter 8	Future scope	22
	Appendix	23
	Bibliography	23

List of Figures

Figure No.	Title
Fig. 2.1	Overall System Architecture
Fig. 2.2	System Workflow Diagram
Fig. 6.1	User Interface – Query Input Page
Fig 6.2	Dashboard
Fig. 6.3	Docker Desktop
Fig. 6.4	Program Files

List of Tables

Table No.	Title
Table 3.1	API Endpoints Description
Table 5.1	JSON Report Structure
Table 6.1	Performance Analysis Metrics

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
GenAI	Generative Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
UI	User Interface
UX	User Experience
CI/CD	Continuous Integration / Continuous Deployment
EC2	Elastic Compute Cloud
FAISS	Facebook AI Similarity Search
NLP	Natural Language Processing
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation

Chapter 1

Introduction

1.1 Introduction

In the modern digital era, the volume of information available on the internet has grown exponentially. While this abundance of data provides immense opportunities for knowledge discovery, it also introduces challenges in efficiently extracting, analyzing, and summarizing relevant information. Traditional research methods require significant manual effort, time, and domain expertise to gather and process information from multiple sources.

To address these challenges, the Energy Research App is developed as an intelligent, AI-powered system that automates the process of web-based research and report generation. The application leverages Generative Artificial Intelligence (Gen AI), Agentic AI (multi-agent systems), and modern web technologies to simulate the workflow of a human research analyst. Additionally, the system is deployed using Docker containers, automated through CI/CD pipelines using GitHub Actions, and hosted on AWS EC2, demonstrating real-world DevOps practices.

The system accepts a user-defined query related to energy or general research topics and performs the following operations:

- Searches the web for relevant information
- Scrapes content from multiple sources
- Filters and analyzes the collected data
- Generates a structured research report using a Large Language Model (LLM)
- Stores the results for future retrieval and semantic search

1.2 Literature Review

Traditional search engines such as Google provide access to vast amounts of data but require users to manually filter, interpret, and compile information.

More recently, AI-based tools such as conversational agents and research assistants have emerged. These systems utilize Large Language Models (LLMs) to generate human-like responses and summaries. However, most existing solutions exhibit certain limitations:

- Lack of end-to-end automation of the research pipeline
- Limited ability to scrape and analyze real-time web data
- Absence of multi-agent collaboration mechanisms
- Minimal support for structured report generation
- No persistent storage and semantic retrieval of research outputs

Research in Agentic AI systems has introduced the concept of dividing complex tasks into smaller subtasks handled by specialized agents.

1.3 Objectives

The primary objectives of this project are as follows:

- To design and develop an automated system for web-based research
- To implement a multi-agent AI architecture for task distribution
- To integrate Generative AI models for report generation
- To enable semantic search using vector embeddings
- To develop a user-friendly interface for interaction and visualization
- To deploy the system using Docker and AWS cloud infrastructure
- To implement a CI/CD pipeline for continuous integration and deployment

1.4 Significance

The significance of this project lies in its ability to combine multiple advanced technologies into a single functional system. The key contributions include:

- Automation of Research Process: Reduces manual effort and time required for gathering and analyzing information
- Integration of AI Technologies: Demonstrates practical use of Generative AI and Agentic AI
- Scalable Architecture: Designed using modular components for easy scalability and maintenance
- Real-World Deployment: Implements industry-standard DevOps practices such as containerization and CI/CD

1.5 Research Design

The system follows a modular and layered design approach, ensuring separation of concerns and ease of development.

Design Approach:

- Frontend Layer: Handles user interaction and displays results
- API Layer: Acts as a communication bridge between frontend and backend
- Backend Layer: Processes requests and manages AI operations
- Agent Layer: Executes specialized tasks such as research, analysis, and summarization
- Storage Layer: Stores reports and embeddings

1.6 Source of Data

The data used in this system is collected dynamically from online sources. The primary sources include:

- Search Engines: DuckDuckGo for retrieving relevant links

- Web Pages: Scraped using BeautifulSoup and HTTP requests
- Fallback Sources: Wikipedia for reliable information when search fails

The collected data is processed and filtered before being passed to the AI models for analysis and summarization.

1.7 Chapter Scheme

The report is organized into multiple chapters, each focusing on a specific aspect of the project:

- Chapter 1: Introduction, objectives, and overview of the system
- Chapter 2: System architecture, design, and workflow
- Chapter 3: Implementation details of frontend, backend, and AI agents
- Chapter 4: DevOps practices including Docker, CI/CD, and deployment
- Chapter 5: Database and storage mechanisms
- Chapter 6: Results and discussion of system performance
- Chapter 7: Summary and conclusions
- Chapter 8: Future scope and enhancements

Chapter 2

System Design & Investigation

2.1 System Architecture

The AI-Powered Legal Intelligence System is designed using a modular, scalable, and layered architecture that integrates frontend interaction, backend processing, artificial intelligence modules, and deployment infrastructure. The architecture ensures separation of concerns, flexibility, and ease of maintenance.

The system is divided into the following major layers:

1. Presentation Layer (Frontend)

The presentation layer is responsible for user interaction and visualization of results.

Key Responsibilities:

- Accept user queries and document uploads
- Display responses generated by the system
- Show research progress and history
- Provide an intuitive and user-friendly interface

The frontend communicates with the backend through HTTP APIs and ensures smooth user experience.

2. Application Layer (API Server)

The application layer acts as an intermediary between the frontend and backend systems.

Key Responsibilities:

- Handles API requests from the frontend
- Routes requests to the appropriate backend services
- Manages security, CORS, and request validation
- Ensures proper communication between different components

This layer ensures that the system remains modular and scalable.

3. Intelligence Layer (AI Backend)

This is the core processing layer where all AI-related computations are performed.

The system integrates:

- Generative Artificial Intelligence for document interaction
- Agentic AI for autonomous legal research

Key Components:

- Document processing module

- Embedding generation
- Query processing engine
- Response generation using LLMs

This layer is responsible for understanding user queries, retrieving relevant information, and generating accurate responses.

4. Agent Layer (Multi-Agent System)

The system uses a multi-agent architecture where each agent performs a specialized task.

Agents Included:

- **Research Agent:** Searches legal information from various sources
- **Scraper Agent:** Extracts relevant content from web pages
- **Analysis Agent:** Filters and processes extracted data
- **Summary Agent:** Generates structured responses using AI models

This modular design improves efficiency, scalability, and accuracy.

5. Data Layer

The data layer is responsible for storing and retrieving structured and unstructured data.

Components:

- Vector database for semantic search
- JSON-based storage for reports and history

The system uses embeddings to enable intelligent search rather than simple keyword matching.

6. Deployment Layer

The deployment layer ensures that the system is accessible, scalable, and production-ready.

Key Features:

- Containerization using Docker
- Automated deployment using CI/CD pipeline
- Cloud hosting on AWS EC2

This layer enables continuous integration and delivery of updates without manual intervention.

AI-Powered Legal Intelligence System

Overall System Architecture

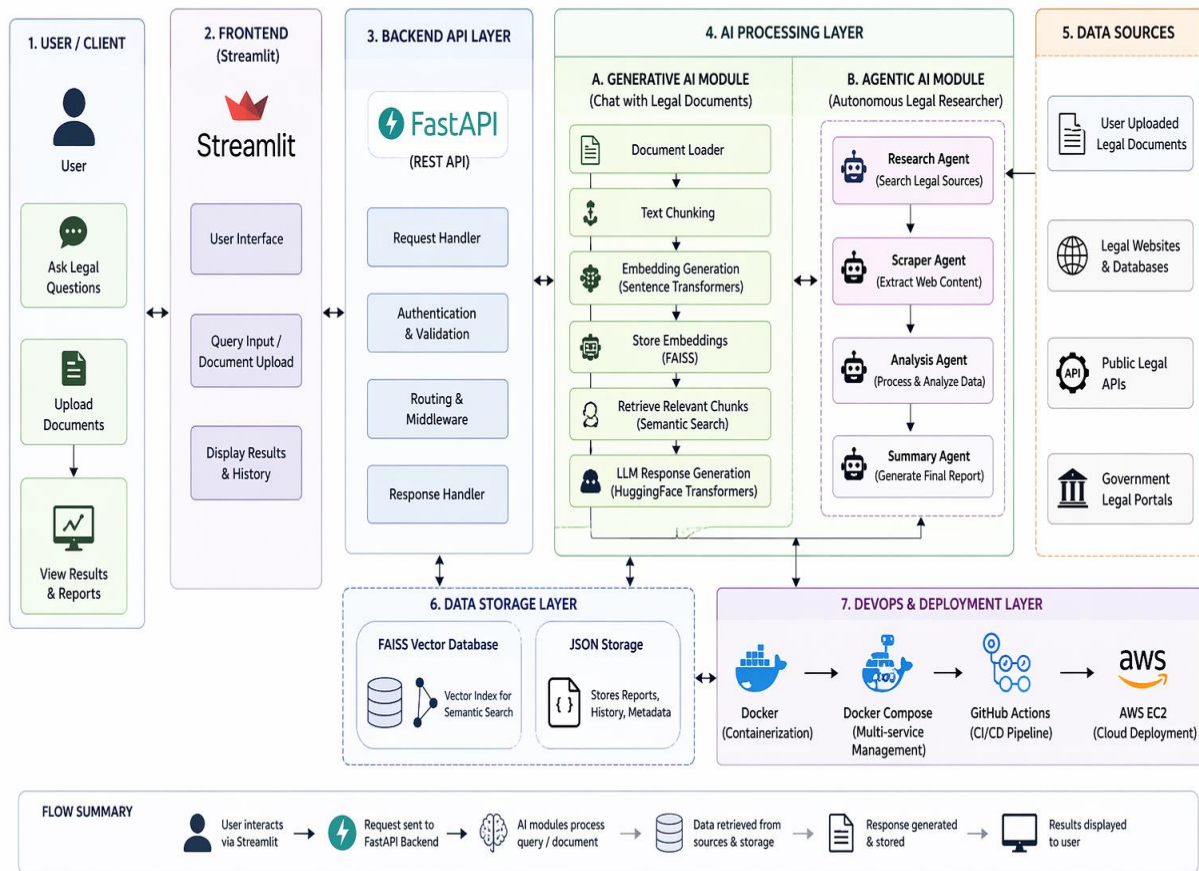


Fig. 2.1: Overall System Architecture

2.2 Technology Stack

The system integrates a wide range of modern technologies across different layers to ensure high performance, scalability, and ease of development.

1. Frontend Technologies

- **Streamlit:**
Used to build an interactive and user-friendly web interface for the application. It allows rapid development of data-driven applications with minimal code.
 - Handles user input (queries and document upload)
 - Displays generated responses and research results
 - Enables quick prototyping and deployment
- **HTML/CSS (optional customization):**
Used for enhancing UI design and styling within Streamlit components

2. Backend Technologies

- **Node.js (Express):**
 - Acts as API gateway
 - Handles routing and communication

- FastAPI (Python):
 - Handles AI processing
 - Executes backend logic efficiently

3. Artificial Intelligence & Machine Learning

- HuggingFace Transformers:
 - Used for text generation and summarization
- Sentence Transformers:
 - Used for generating embeddings
- Natural Language Processing:
 - Enables understanding of legal text

4. Data Processing Tools

- BeautifulSoup:
 - Used for web scraping
- Requests Library:
 - Handles HTTP requests

5. Database & Storage

- FAISS (Vector Database):
 - Enables semantic search
 - Efficient similarity matching
- JSON Storage:
 - Stores structured legal reports
 - Maintains history

6. DevOps & Deployment Tools

- Docker:
 - Containerizes the application
- Docker Compose:
 - Manages multiple services
- GitHub Actions:
 - Implements CI/CD pipeline
- AWS EC2:
 - Hosts the application on cloud

Technology Stack Summary

The combination of frontend, backend, AI, and DevOps tools ensures:

- High performance
- Scalability
- Efficient deployment
- Real-world applicability

2.3 System Workflow

The system follows a structured workflow that automates the entire process from user input to final output generation.

Step 1: User Input

- User uploads a legal document or enters a query
- Input is received through the frontend interface

Step 2: API Request Handling

- Request is sent to the API server
- API validates and forwards request to backend

Step 3: Document Processing (Gen AI Module)

- Uploaded document is processed
- Text is extracted and cleaned
- Converted into embeddings
- Stored in vector database

Step 4: Query Processing

- User query is analyzed
- Relevant document chunks are retrieved using semantic search

Step 5: Agent Execution (Agentic AI Module)

The multi-agent system is activated:

1. Research Agent
 - Searches for relevant legal data
2. Scraper Agent
 - Extracts useful information
3. Analysis Agent
 - Filters and organizes data
4. Summary Agent
 - Generates structured output

Step 6: Response Generation

- LLM generates final response
- Response includes:
 - Explanation
 - Key insights
 - Summary

Step 7: Storage & Indexing

- Results stored in JSON format
- Embeddings stored for future search

Step 8: Response Delivery

- Backend sends response to frontend
- User views final output

Workflow Summary

The system workflow ensures:

- Automation of legal research
- Efficient document interaction
- Fast and accurate response generation

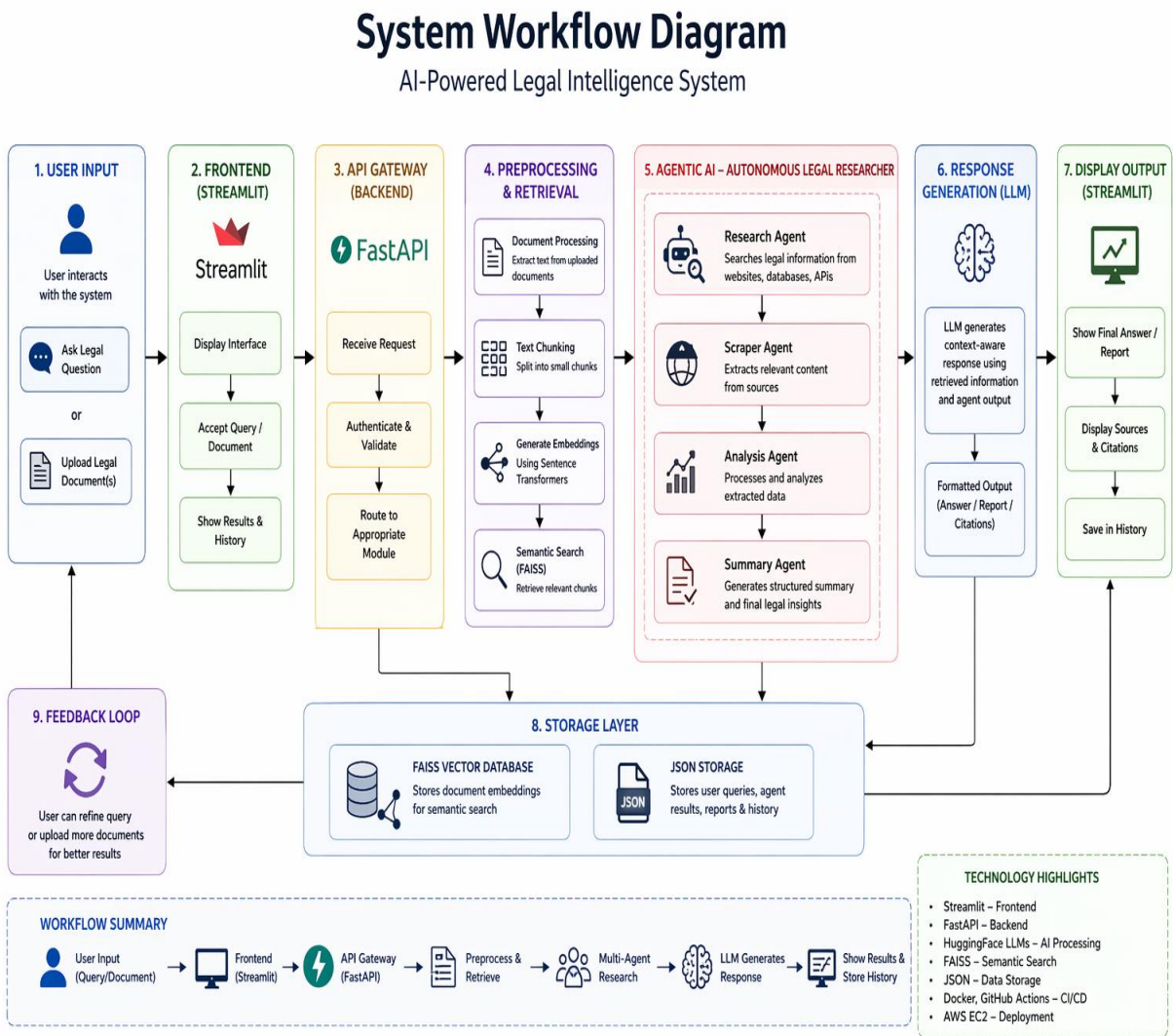


Fig. 2.2: System Workflow Diagram

Chapter 3

Implementation

3.1 Frontend Implementation

The frontend of the AI-Powered Legal Intelligence System is developed using Streamlit, which provides a simple and efficient way to build interactive web applications using Python. Streamlit enables rapid development of user interfaces without requiring extensive frontend frameworks, making it suitable for AI-driven applications.

The interface is designed to be clean, user-friendly, and responsive, allowing users to easily interact with the system. It supports functionalities such as entering legal queries, uploading documents, and viewing generated responses in real time.

3.1.1 User Interface Features

The frontend includes the following key features:

- **Query Input Interface:**
Users can enter legal queries in natural language to retrieve relevant information.
- **Document Upload Feature:**
Allows users to upload legal documents such as contracts and agreements for analysis.
- **Results Display:**
Displays AI-generated responses, summaries, and insights in a structured format.
- **History Section:**
Stores and displays previous queries and generated outputs for easy reference.

3.1.2 API Integration

The Streamlit frontend communicates with backend services using HTTP requests. The integration is handled using Python libraries, enabling smooth interaction between the frontend and backend components.

- **API Communication:**
All API calls are made using:
 - **Requests Library:** For sending HTTP GET and POST requests
 - **JSON-based request and response handling**
- **Request Handling Flow:**
 1. User submits query or uploads document
 2. Streamlit sends request to backend API
 3. Backend processes request using AI models
 4. Response is returned and displayed on UI

3.1.3 Advantages of Using Streamlit

- Simplified frontend development using Python
- Faster prototyping and deployment
- Seamless integration with AI and backend systems
- Minimal setup compared to traditional frontend frameworks

3.2 Backend Implementation

The backend consists of two layers:

1. API Server (Node.js)
2. AI Backend (FastAPI)

3.2.1 API Server (Node.js - Express)

The API server acts as a gateway between frontend and AI backend.

Responsibilities:

- Route /api/* requests
- Forward to Python backend
- Handle CORS issues
- Maintain separation of concerns

Example: /api/research → forwarded to → Python /research

3.2.2 Python Backend (FastAPI)

The Python backend handles all AI processing and business logic.

Key Modules:

- main.py → Entry point
- agents/ → AI agents
- tools/ → Search & scraping utilities
- database/ → Storage & FAISS
- models/ → LLM integration

3.2.3 API Endpoints Implemented

Method	Endpoint	Description
GET	/healthz	Health check
POST	/research	Start research
GET	/research/{id}	Get result
GET	/history	Fetch reports
POST	/search	Semantic search

3.3 AI Agent Implementation

The core intelligence of the system is based on a multi-agent architecture, where each agent performs a specific task in the research pipeline.

3.3.1 Research Agent

- Performs web search using DuckDuckGo
- Collects top URLs
- Initiates scraping process

3.3.2 Scraper Tool

- Uses BeautifulSoup
- Extracts:
 - Paragraphs
 - Headings
 - Relevant content
- Handles multiple URLs concurrently

3.3.3 Analysis Agent

- Filters irrelevant data
- Extracts key insights
- Identifies main topics

3.3.4 Summary Agent

- Uses HuggingFace Flan-T5 model
- Generates structured reports:
 - Introduction
 - Key insights
 - Conclusion

3.3.5 Embedding & Storage

- Sentence Transformers generate embeddings
- Stored in FAISS for semantic search
- Reports saved in JSON format

3.3.6 Agent Pipeline Flow

The agents operate sequentially:

1. Research Agent → gathers data
2. Scraper → extracts content
3. Analysis Agent → processes data
4. Summary Agent → generates report

Chapter 4

DevOps & Deployment

4.1 Dockerization

To ensure portability, consistency, and scalability, the entire application is containerized using Docker. Each major service runs inside its own isolated container.

4.1.1 Multi-Container Architecture

The system is divided into three Docker services:

- Frontend Container
 - Built using React + Vite
 - Served via Nginx
 - Runs on port 80
- API Server Container
 - Node.js (Express)
 - Handles routing and proxy logic
 - Runs on port 8080
- Python Backend Container
 - FastAPI-based AI engine
 - Executes multi-agent pipeline
 - Runs on port 8000

4.1.2 Dockerfile Strategy

Each service has its own optimized Dockerfile:

Frontend

- Multi-stage build:
 - Build stage → Node.js (pnpm build)
 - Production stage → Nginx (serve static files)

Backend

- Lightweight Python image
- Installs dependencies via requirements.txt
- Runs FastAPI using Uvicorn

4.1.3 Docker Compose Orchestration

All services are managed using **Docker Compose**, enabling:

- Service networking
- Environment isolation
- Easy startup/shutdown

Example:

services:

frontend:

api-server:

python-backend:

4.1.4 Benefits of Dockerization

- Environment consistency (works on any system)
- Easy deployment on cloud
- Isolation of services
- Simplified dependency management

4.2 CI/CD Pipeline (GitHub Actions)

The project implements a **Continuous Integration and Continuous Deployment (CI/CD)** pipeline using **GitHub Actions**.

4.2.1 Pipeline Overview

The pipeline automates:

- Code integration
- Docker image building
- Image publishing
- Remote deployment

4.2.2 Workflow Steps

1. Developer pushes code to GitHub (main branch)
2. GitHub Actions is triggered
3. Docker images are built:
 - energy-frontend
 - energy-api-server
 - energy-python-backend
4. Images are pushed to Docker Hub
5. EC2 instance is accessed via SSH
6. Latest images are pulled

7. Containers are restarted using Docker Compose

4.2.3 Deployment Script Logic

Key operations:

```
docker pull <images>
```

```
docker-compose down
```

```
docker-compose up -d
```

```
docker image prune -af
```

4.2.4 Key Features of Pipeline

- Fully automated deployment
- Zero manual server intervention
- Consistent production updates
- Integrated cleanup (prevents disk overflow)

4.3 AWS EC2 Deployment

The application is deployed on a cloud-based virtual server using AWS EC2, enabling remote access and real-time usage.

4.3.1 EC2 Instance Setup

- OS: Ubuntu Linux
- Docker & Docker Compose installed
- Security groups configured:
 - Port 80 → Frontend
 - Port 8080 → API Server
 - Port 8000 → Backend

4.3.2 Deployment Process

1. EC2 instance is launched
2. SSH access configured using key pair
3. Docker environment installed
4. Application deployed via CI/CD pipeline
5. Services started using Docker Compose

4.3.3 Application Access

- Frontend accessed via:

`http://<EC2-PUBLIC-IP>`

- API server:

http://<EC2-PUBLIC-IP>:8080

4.3.4 Advantages of Cloud Deployment

- Global accessibility
- Scalability
- High availability
- Real-time updates via CI/CD

4.4 Summary of DevOps Implementation

The integration of Docker, CI/CD, and AWS enables:

- Automated deployment pipeline
- Scalable architecture
- Reduced manual errors
- Faster development-to-production cycle

Chapter 5

Database & Storage

5.1 FAISS Vector Database

The system uses FAISS as a vector database to enable semantic search over stored research reports. Traditional databases rely on keyword matching, which is limited. FAISS allows:

- Semantic similarity search
- Faster retrieval of relevant information
- Efficient handling of high-dimensional embeddings

5.1.1 Working Principle

1. Text data is converted into vector embeddings using Sentence Transformers
2. These embeddings are stored in FAISS index
3. When a query is received:
 - It is converted into a vector
 - FAISS finds closest matching vectors using similarity search

5.1.2 Advantages

- High-speed similarity search
- Scalable for large datasets
- Enables intelligent search instead of keyword matching

5.2 JSON Storage System

In addition to FAISS, the system uses a JSON-based storage mechanism for storing complete research reports.

- Store structured research outputs
- Maintain history of generated reports
- Provide quick retrieval without recomputation

5.2.1 Storage Structure

Each report is stored as a JSON file containing: Report ID, Query, Generated summary, Sources, Timestamp

Example structure:

```
{  
  "id": "legal_report_001",  
  "query": "What are the termination clauses in this contract?",
```

"response": "The contract includes termination clauses such as mutual agreement, breach of contract, and expiration of term.",

"summary": "Termination can occur through mutual consent, violation of terms, or completion of contract duration.",

"sources": [

"uploaded_document: contract_sample.pdf",

"section: clause 5 - termination"

],

"document_name": "contract_sample.pdf",

"timestamp": "2026-04-24 14:30:00"

}

5.2.3 Advantages

- Lightweight and easy to manage
- Human-readable format
- No need for complex database setup
- Suitable for small-to-medium scale systems

5.3 Data Flow Management

The system ensures efficient data flow between different components, from user input to final output storage and retrieval.

5.3.1 End-to-End Data Flow

1. User submits query
2. Backend processes request via agents
3. Data is: Stored as JSON (report), Converted to embeddings (vector)
4. Stored in: JSON storage (for retrieval), FAISS index (for semantic search)

5.3.2 Data Lifecycle

- Input Stage → Query received
- Processing Stage → Agents analyze data
- Storage Stage → JSON + FAISS
- Retrieval Stage → API endpoints fetch data

5.3.3 API Integration

Data is accessed via endpoints:

- /history → Fetch all reports

- /history/{id} → Fetch specific report
- /search → Semantic search using FAISS

5.3.4 Synchronization Between Storage Systems

- Each JSON report has a corresponding embedding
- FAISS stores vector references mapped to report IDs
- Ensures consistency between:
 - Structured data (JSON)
 - Semantic data (FAISS)

Chapter 6

Results and Discussions

6.1 System Output

The AI-Powered Legal Intelligence System successfully generates accurate and structured responses for legal queries using its Generative AI and Agentic AI modules. The system enables users to interact with legal documents and perform autonomous legal research efficiently.

6.1.1 Output Generation

When a user submits a query or uploads a legal document:

- The system processes the input using Generative AI and multi-agent architecture
- Relevant information is retrieved from uploaded documents or external sources
- Semantic search is performed using vector embeddings
- The system generates a structured response containing:
 - Legal Explanation
 - Key Clauses / Insights
 - Summarized Output
 - References / Sources

This automated pipeline significantly reduces manual effort in legal analysis.

6.1.2 Sample Output Description

For example, for the query:

“What are the termination clauses in a contract?”

The system produces:

- Definition of termination clauses
- Types of termination (mutual, breach, expiration)
- Explanation of conditions and legal implications
- Extracted clauses (if document is uploaded)
- References from document sections or external sources

This demonstrates the system’s ability to provide context-aware and domain-specific responses.

6.1.3 Frontend Output Display

The frontend, built using Streamlit, displays results in a user-friendly format.

Displayed Features:

- Query input and document upload interface
- Generated legal responses in structured format
- Extracted insights and summaries
- Source references and document sections
- History of previous queries and results

The interface ensures easy interaction and clear visualization of outputs.

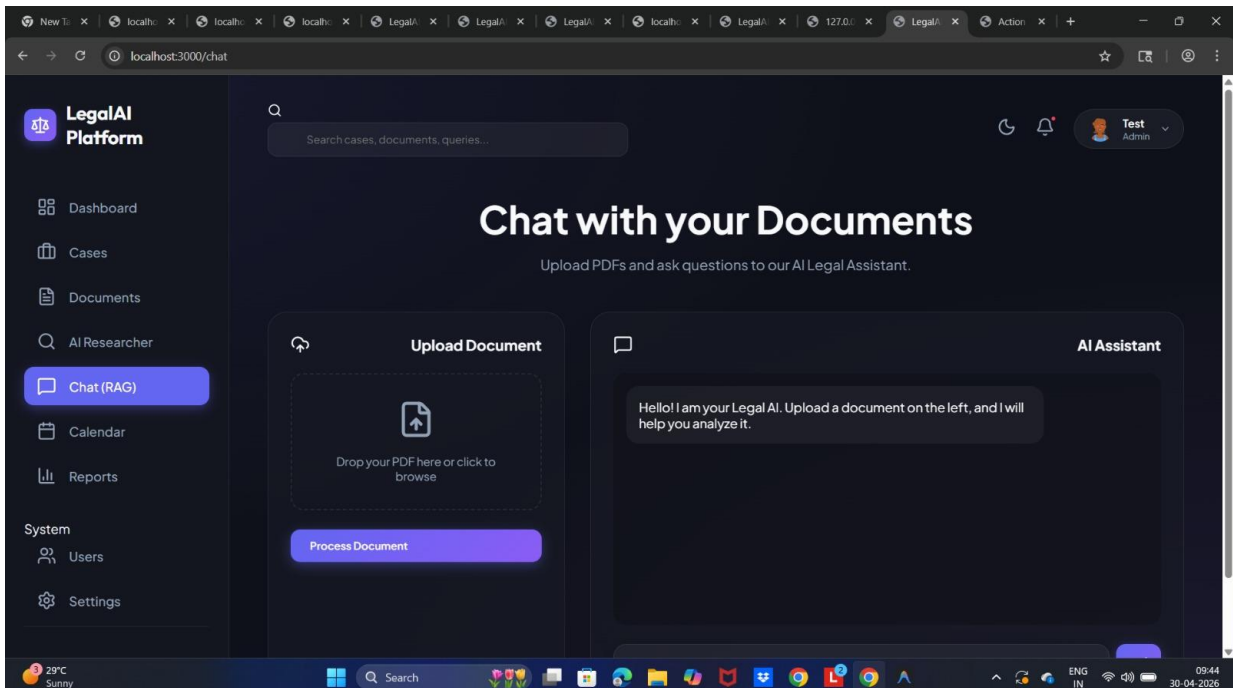
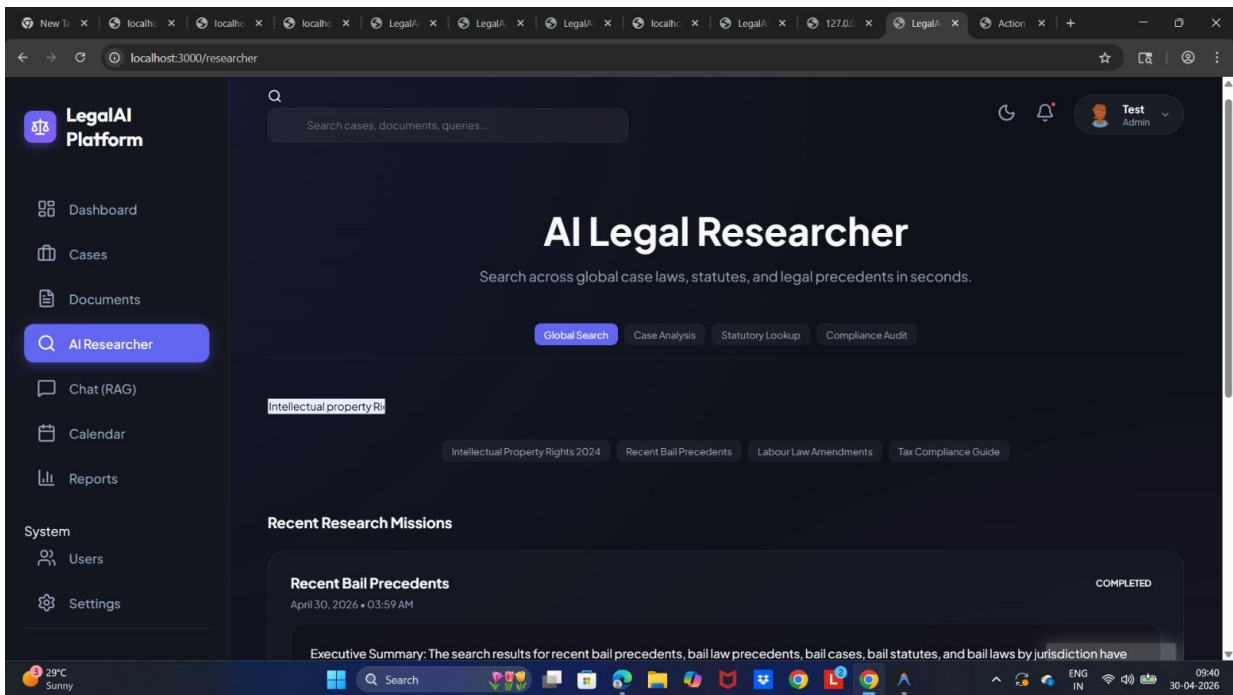


Fig. 6.1: User Interface – Query Input Page

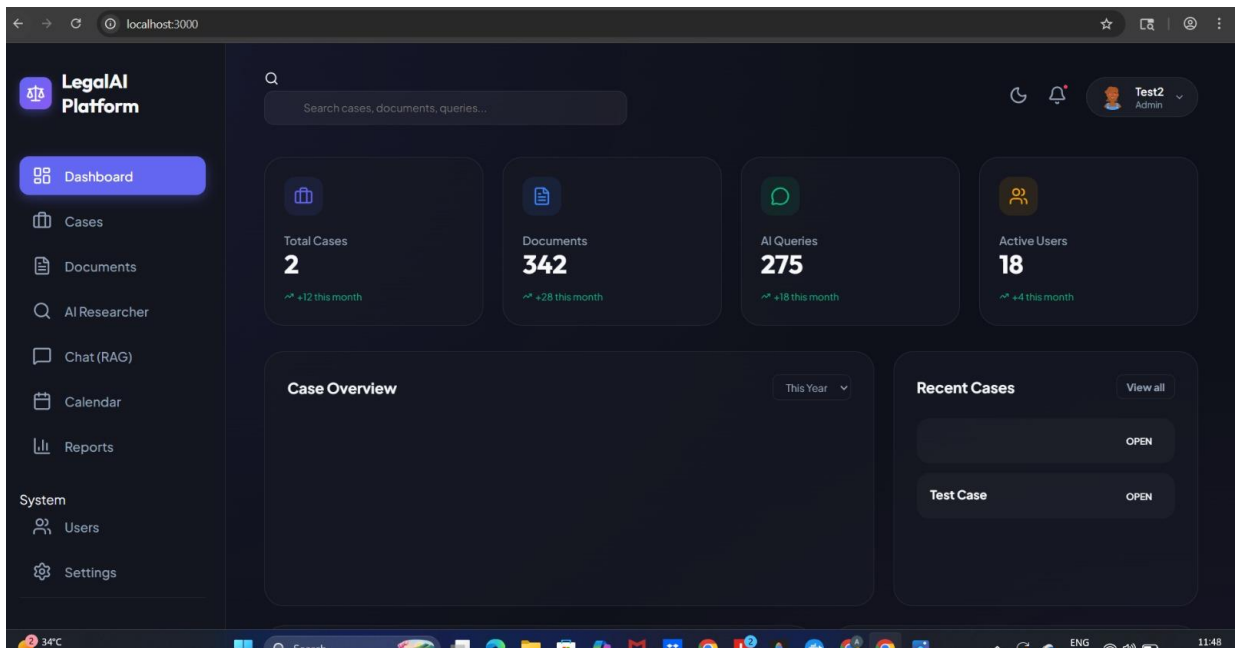


Fig 6.2: Dashboard

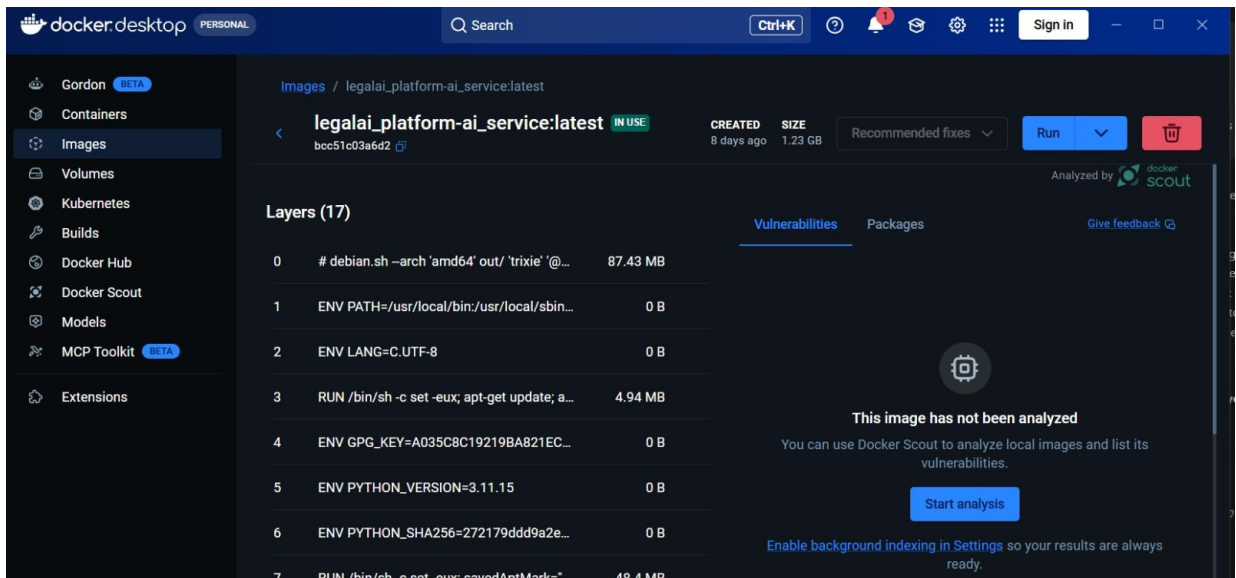


Fig 6.3: Docker Desktop

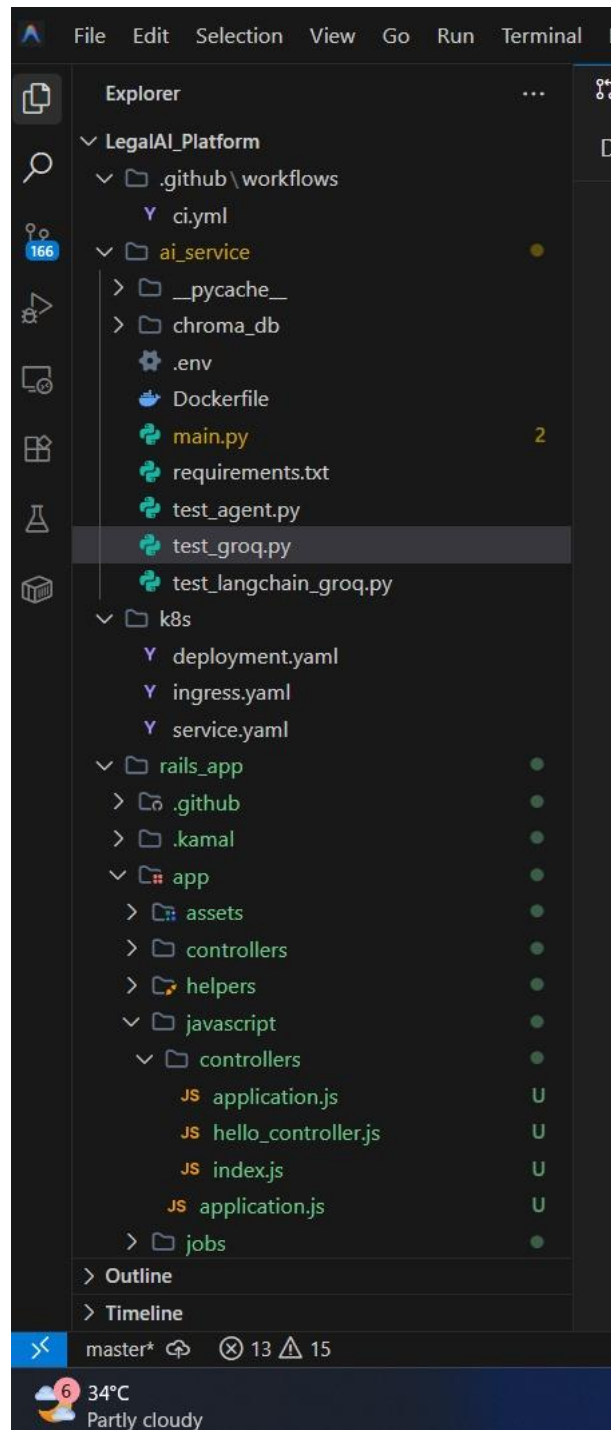


Fig 6.4: Program files

6.1.4 Observations

- System successfully integrates multiple components
- Reports are generated in structured format
- Real-time updates improve user experience

6.2 Performance Analysis

The performance of the system was evaluated based on response time, accuracy, and system efficiency.

6.2.1 Response Time

Observation:

- Most delay occurs in: Web scraping, LLM processing

6.2.2 System Efficiency

- Concurrent scraping improves speed
- Docker ensures consistent performance
- CI/CD reduces deployment time

6.2.3 Scalability

- Docker-based architecture allows scaling
- Can handle multiple users with container scaling
- FAISS supports large dataset search

6.3 Challenges Faced

During development and deployment, several technical challenges were encountered.

6.3.1 API Routing Issues

Problem: 404 errors when frontend tried to access backend

Cause: Incorrect base URL and missing /api routing

Solution: Fixed API proxy configuration, Corrected base URL in frontend

6.3.2 Deployment Issues on Cloud

Problem: Application worked locally but not on EC2

Cause: Environment mismatch and port configuration

Solution: Updated Docker configuration, Corrected exposed ports and URLs

6.3.3 Disk Space Issues

Problem: EC2 storage getting full due to multiple images

Solution: Implemented cleanup command: `docker image prune -af`

Chapter 7

Summary and Conclusion

7.1 Summary

This project presented the design and implementation of an AI-Powered Legal Intelligence System that automates the process of legal document analysis, information retrieval, and research.

The system integrates advanced technologies such as Generative Artificial Intelligence and Agentic AI to enhance efficiency and accuracy in legal workflows.

The application follows a multi-agent architecture, where:

- Research Agent collects relevant legal information from various sources
- Scraper Agent extracts useful content from documents and web pages
- Analysis Agent processes and filters the extracted data
- Summary Agent generates structured and meaningful legal responses

Additionally, the system incorporates:

- Semantic search using vector embeddings for accurate information retrieval
- Document-based interaction using Generative AI
- Web scraping techniques for real-time legal data extraction
- LLM-based response generation using transformer models
- A user-friendly interface built with Streamlit
- Cloud deployment on Amazon EC2 for accessibility
- Automated deployment using CI/CD pipelines

The system successfully demonstrates how AI technologies can be applied to simplify complex legal tasks.

7.2 Conclusion

The AI-Powered Legal Intelligence System successfully demonstrates the integration of Artificial Intelligence, Agentic Systems, and DevOps practices into a unified and scalable application.

Key conclusions drawn from the project:

- The system is capable of generating accurate, context-aware, and structured legal responses
- The multi-agent architecture improves modularity, flexibility, and efficiency
- Semantic search using FAISS significantly enhances information retrieval accuracy
- Generative AI enables natural interaction with legal documents
- Containerization using Docker and CI/CD automation using GitHub Actions ensure smooth and reliable deployment
- The system is scalable and suitable for real-world applications

Despite minor challenges during development, such as API integration issues, deployment configurations, and model performance limitations, the final system operates reliably and efficiently.

Overall, the project showcases a modern AI-driven architecture that combines:

- Machine Learning and Natural Language Processing
- Web-based application development
- Cloud deployment
- Automation through DevOps practices

This system provides a strong foundation for future advancements in AI-powered legal solutions and intelligent research systems.

Chapter 8

Future Scope

The current system provides a strong foundation for automated legal document analysis and research. However, several enhancements can be implemented to further improve functionality, accuracy, and scalability.

1. Advanced AI Models

- Integration of more powerful Large Language Models such as GPT-4 or newer transformer-based models
- Improved contextual understanding of complex legal language
- Enhanced summarization and reasoning capabilities for legal documents

2. Real-Time Legal Data Integration

- Integration with live legal APIs and databases
- Access to real-time case laws, judgments, and regulations
- Integration with legal research platforms and online repositories

3. User Authentication System

- Secure login and registration system
- Role-based access (admin, user, legal professional)
- Personalized dashboards with user-specific history and saved documents

4. Scalability Improvements

- Deployment using Kubernetes for advanced container orchestration
- Load balancing for handling multiple concurrent users
- Microservices-based scaling for better performance

5. Enhanced Semantic Search

- Hybrid search combining keyword-based and vector-based retrieval
- Improved ranking and relevance algorithms
- Context-aware search results for better accuracy

6. UI/UX Improvements

- Advanced and interactive dashboards
- Visualization of legal insights using charts and graphs
- Improved user experience in Streamlit interface

7. Mobile Application Development

- Development of Android and iOS applications
- Cross-platform support using modern frameworks
- Mobile-friendly interface for accessibility

8. Integration with External Tools

- Integration with legal citation tools
- Export reports in formats such as PDF, DOCX
- Integration with document management systems

9. Performance Optimization

- Implementation of caching mechanisms for faster response
- Optimization of AI models for reduced latency
- Efficient document processing and indexing

10. Security Enhancements

- Implementation of JWT-based API authentication
- Secure storage and encryption of legal data
- Protection against unauthorized access and data breaches

Appendix

Appendix A: Sample API Endpoints

Method	Endpoint	Description
GET	/health	Health check for backend service
POST	/upload	Upload legal documents
POST	/query	Query uploaded legal documents
POST	/research	Start autonomous legal research
GET	/research/{id}	Retrieve research results
GET	/history	Fetch previous queries and reports
POST	/search	Perform semantic search

Appendix B: Sample Docker Commands

```
docker compose up -d
docker compose down
docker image prune -af
```

Appendix C: Project Structure

```
project-root/
├── frontend-streamlit/
├── backend-fastapi/
├── agents/
├── database/
├── docker/
└── ci-cd/
```

Bibliography

- [1] Tom B. Brown et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems*, 2020.
- [2] Docker Inc., “Docker Documentation,” Available: <https://docs.docker.com/>
- [3] Amazon Web Services, “Amazon EC2 Documentation,” Available: <https://docs.aws.amazon.com/ec2/>
- [4] Hugging Face, “Transformers Documentation,” Available: <https://huggingface.co/docs>
- [5] Jacob Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL*, 2019.
- [6] Ashish Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [7] FAISS, “FAISS Documentation,” Available: <https://faiss.ai/>
- [8] Streamlit, “Streamlit Documentation,” Available: <https://docs.streamlit.io/>
- [9] FastAPI, “FastAPI Documentation,” Available: <https://fastapi.tiangolo.com/>
- [10] Node.js Foundation, “Node.js Documentation,” Available: <https://nodejs.org/>
- [11] GitHub, “GitHub Actions Documentation,” Available: <https://docs.github.com/actions>
- [12] OpenAI, “GPT Models Documentation,” Available: <https://platform.openai.com/docs>
- [13] Natural Language Processing – Jurafsky, D., & Martin, J. H., *Speech and Language Processing*, 3rd Edition.
- [14] Elastic, “Elasticsearch Guide,” Available: <https://www.elastic.co/guide>
- [15] Kubernetes, “Kubernetes Documentation,” Available: <https://kubernetes.io/docs/>